COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	07/28/26	**DATE SUBMITTED:**	07/28/26
NAME:	Medina, Jomz Ryle E.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 **Stacks**

1. PROCEDURE OUTPUT

What procedures did you perform?

Welcome | main.cpp | stack

main.cpp > main()

```cpp
#include<iostream>
#include<stack> // calling stack from the STL

int main(){
    // create an object for the stack
    std::stack<int> newStack;

    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;

    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;
    std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
    std::cout<<"The top of the element is: "<< newStack.top()<<std::endl
    std::cout<<"Removing elements from the stack: \n";
    newStack.pop();
    std::cout<<"The top of the element is: "<< newStack.top()<<std::endl
    newStack.pop();
    std::cout<<"The top of the element is: "<< newStack.top()<<std::endl
    newStack.pop();
    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;
    return 0;
}
```

```
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA4p1> cd "c:\Users\TIPQC\Documents\ME
PE010 - CPE21S1\HOA4p1\" ; if ($?) { g++ main.cpp -o main } ; if ($?) { .\main }
Stack Empty? 1
Stack Empty? 0
The size of the stack: 3
The top of the element is: 15
Removing elements from the stack:
The top of the element is: 8
The top of the element is: 3
Stack Empty? 1
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA4p1>
```

Code Analysis:

The code above shows the use of stack operations, included via inserting the *stack* header library. It shows basic functions such as checking if the created stack is empty, finding the size, as well as adding and removing via top and pop.

Table 4-1. Output of ILO A

Header, global variables, function prototype & main

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();
void display();

int main() {
    int choice;
    std::cout<<"Enter the number of max elements for the new stack: ";
    std::cin>>i;

    while(true){
        std::cout<<"---Stack Operations---" << std::endl;
        std::cout<<"1. PUSH\n2. POP\n3. TOP\n4. isEMPTY\n5. Display\n";
        std::cin>>choice;
        std::cout<<std::endl;
        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout<< isEmpty() <<std::endl;
            break;
            case 5: display();
            break;
            default: std::cout<< "Invalid choice!\n";
        }
    }

    return 0;
}
```

isEmpty()

```cpp
bool isEmpty(){
    // create a condition to check whether the top element is equal to -1
    if(top == -1) return true;
    return false;
}
```

The code above shows the isEmpty() function which checks if the stack is empty by checking the 'index' starting from -1.

isFull()

```cpp
bool isFull(){
    // check if we reach the maximum capacity of the array
    if(top >= i-1) return true;
    return false;
}
```

This code checks if the stack is full, based on the given value from the user as variable, 'i'. It does this by comparing the position of top to the size.

push()

```
54  void push(){
55      // check if full -> if yes, return error
56      if(isFull()){
57          std::cout<<"Stack overflow!\n";
58          return;
59      }
60
61      std::cout << "New value to add into the stack: ";
62      std::cin >> newData;
63      // increment the top (index by 1) then assign the new data
64      stack[++top] = newData;
65      std::cout<<std::endl;
66  }
```

This function is what we use for the main adding of an element to the stack. It has a check if the stack is already full. Additionally, top is actually 'pre-incremented' (shown by the '++' before top) as doing so, before the actual top allows for adding to top before using the value in the stack array.

Top()

```
68  void Top(){
69      // check if the stack is empty
70      if(isEmpty()){
71          std::cout<<"Stack underflow.\n"<<std::endl;
72      }
73
74      std::cout<<"The element on the top of the stack is " << stack[top]<<std::endl;
75  }
```

This function just shows the top-most element (meaning the newest added one) in the stack. It has a check to verify that you can do the action validly like with the others.

pop()

```
77  void pop(){
78      if(isEmpty()){
79          std::cout<<"Stack underflow.\n"<<std::endl;
80      }
81
82      //display the top value
83      std::cout<<"Popping: " << stack[top];
84      //decrement the top vaue from the stack
85      top--;
86  }
```

This function is mainly used for removing the element at the top, the reverse of **Top()**. As always, it has the check for if the stack is empty as it can't exactly remove, 'nothing'.

display()

```
88  void display(){
89      if(isEmpty()){
90          std::cout << "Stack underflow.\n";
91          return;
92      }
93      std::cout << "Stack (top to bottom): ";
94      for(int j = top; j >= 0; j--){
95          std::cout << stack[j] << " ";
96      }
97      std::cout << std::endl;
98  }
```

For the display function, I actually just used a for-loop to iterate through the entire stack and print the inputted data accordingly.

Terminal:

```
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA4p1\ILO B> cd "
Enter the number of max elements for the new stack: 3
---Stack Operations---
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. Display
1

New value to add into the stack: 3

---Stack Operations---
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. Display
1

New value to add into the stack: 2

---Stack Operations---
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. Display
1


New value to add into the stack: 1

---Stack Operations---
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. Display
5

Stack (top to bottom): 1 2 3
---Stack Operations---
1. PUSH
2. POP
3. TOP
4. isEMPTY
5. Display

Stack (top to bottom):
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA4p1\ILO B>
```

Table 4-2. Output of ILO B.1.

main.cpp

main.cpp stackList.h

HOA5p1 > main.cpp > main()

```cpp
#include <iostream>
#include "stackList.h"

int main(){

    std::cout << "--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---\n";

    // PUSH
    push('c');
    push('p');
    push('e');

    // Display the current top
    Top<char>(); // Top should be 'e'

    std::cout<<"\n --- Displaying the Stack --- \n";
    displayAll<char>();

    std::cout<<"\n --- Popping all the elements --- \n";

    while(!isEmpty<char>()){
        std::cout<<"Popped: " << pop<char>() << std::endl;
    }

    std::cout<<"\n --- Testing the Underflow --- \n";
    pop<char>();

    std::cout<<"\n --- Displaying the Stack --- \n";
    displayAll<char>();

    return 0;
}

```

Terminal Output:

```
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA5p1> cd
--- PUSHING 'c' 'p' 'e' ONTO THE STACK ---
Top of Stack:  e

 --- Displaying the Stack ---
Stack (top to bottom): e -> p -> c

 --- Popping all the elements ---
Popped: e
Popped: p
Popped: c

 --- Testing the Underflow ---
Stack underflow!

 --- Displaying the Stack ---
Stack is empty!
PS C:\Users\TIPQC\Documents\MEDINA CPE010 - CPE21S1\HOA5p1>
```

Code Analysis:

Our **main.cpp** simply shows the functions being called from our very own custom-made stack library for the lab activity. It calls the **push()** function three times to insert **c**, **p**, and **e** into the stack, followed by the displaying of the stack with **displayAll()**. After, we remove the elements with the **pop()** function and test the **isEmpty()** function, properly getting back that the stack is indeed empty. This is further confirmed by re-calling **displayAll()** showing that there are indeed no more elements in the stack.

stackList.h:

Library definition, class, and global variables:

```
#include <iostream>
#ifndef STACK_LIST_H // header file name
#define STACK_LIST_H

template <typename T>
class Node{
    public:
    T data;     // VALUE THAT WE NEED TO STORE
    Node* next; // POINTER TO THE NEXT NODE


};

// Global pointer for the top of the stack
template <typename T>
Node<T>* head = nullptr;

```

This code defines the header files included, the library itself, the creation of the class, and sets up the global variables to be used later.

isEmpty()

```
// ISEMPTY
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}
```

This function simply checks if the stack is empty by seeing if the current head that is pointed at is directed to nullptr (which would otherwise be impossible if it weren't empty).

push()

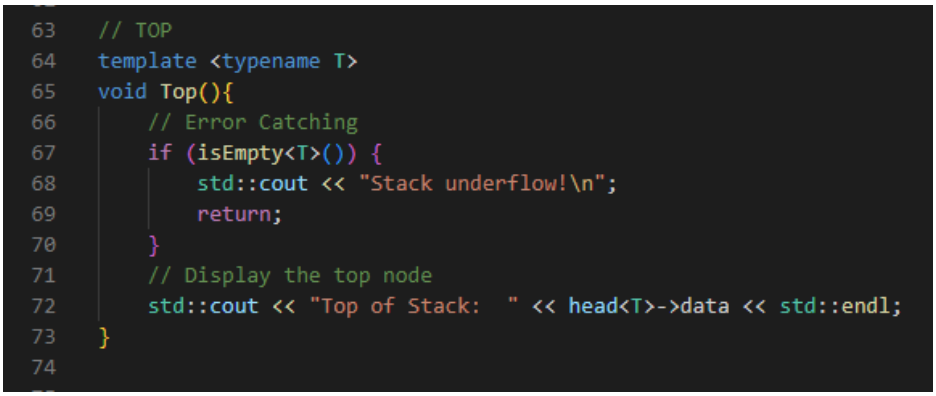
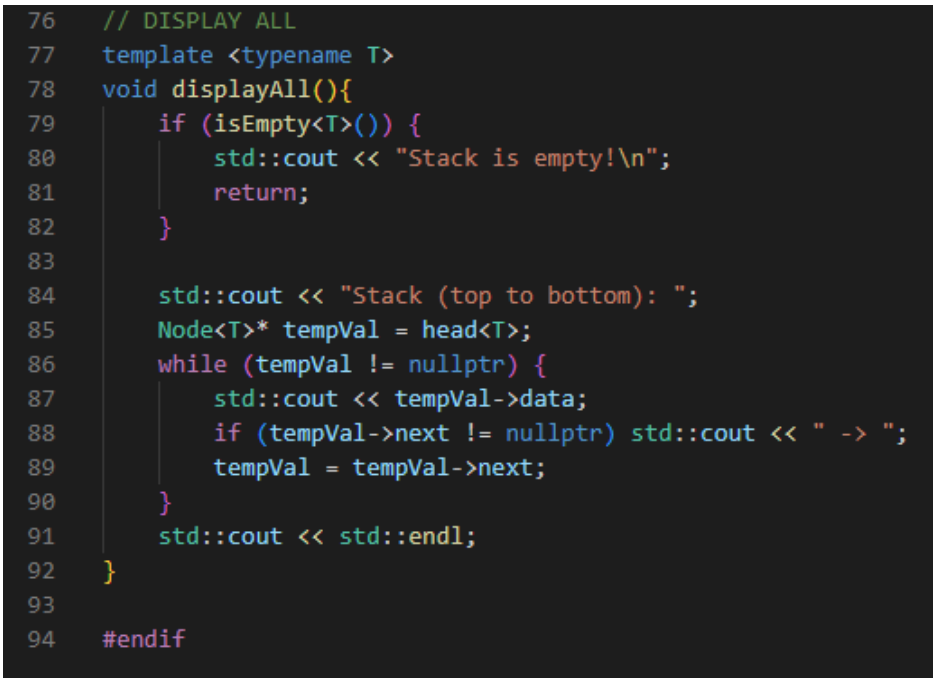
```
27  // PUSH
28  template <typename T>
29  void push(T newData){
30      // Create a new node and assign the newData
31      Node<T>* newNode = new Node<T>();
32      newNode->data = newData;
33
34      // New node points to current top, then becomes the new top
35      newNode->next = head<T>;
36      head<T> = newNode;
37  }
```

This function is what we use for when we need to add elements to the stack. It simply creates the node, inserts its assigned data, then updates the pointers.

pop()

```
39  // POP
40  template <typename T>
41  T pop() {
42      if (isEmpty<T>()) {
43          std::cout << "Stack underflow!\n";
44          return T{};
45      }
46
47      // Save the current top node
48      Node<T>* temp = head<T>;
49
50      // Store its data
51      T tempVal = temp->data;
52
53      // Move the head to the next node
54      head<T> = head<T>->next;
55
56      // Free the removed node
57      delete temp;
58
59      // return the removed value
60      return tempVal;
61  }
62
```

This function is used for removing elements in the stack. It has a check for if the stack is empty first before proceeding with **isEmpty()**, using a temporary variable to store the data, update the pointer, then remove the node, returning the value it removed in the end.

Top()	 This function just displays the most recently added element or the top of the stack. Additionally, it also checks if the stack is empty first before allowing itself to read the top of the stack.
displayAll()	 This function displays all of the elements of the stack, having a check as well with the previous two functions that stops and prints that the stack is empty if it has no elements. It works by storing the head in a temporary variable and using a while loop to print the data accordingly. Until it reaches the last node, where the pointer is directed at nullptr, only then will it stop and finish executing its part.

```
63  // TOP
64  template <typename T>
65  void Top(){
66      // Error Catching
67      if (isEmpty<T>()) {
68          std::cout << "Stack underflow!\n";
69          return;
70      }
71      // Display the top node
72      std::cout << "Top of Stack:  " << head<T>->data << std::endl;
73  }
74
```

```
76  // DISPLAY ALL
77  template <typename T>
78  void displayAll(){
79      if (isEmpty<T>()) {
80          std::cout << "Stack is empty!\n";
81          return;
82      }
83
84      std::cout << "Stack (top to bottom): ";
85      Node<T>* tempVal = head<T>;
86      while (tempVal != nullptr) {
87          std::cout << tempVal->data;
88          if (tempVal->next != nullptr) std::cout << " -> ";
89          tempVal = tempVal->next;
90      }
91      std::cout << std::endl;
92  }
93
94  #endif
```

Table 4-3. Output of ILO B.2.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

ILO C: Solve problems using an implementation of stack:

Raw Code:

mainSupplementary.cpp

```cpp
#include <iostream>
#include "stackList.h"

int main(){
    std::string expressions[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D]}"
    };

    for (const auto& expr : expressions) {
        std::cout << "Expression: " << expr << "\n";
        bool valid = parser(expr);
        std::cout << "Valid? " << (valid ? "Y" : "N") << "\n\n"; // ternary expression
    }

    return 0;
}
```

stackList.h

```
#include <iostream>
#ifndef STACK_LIST_H // header file name
#define STACK_LIST_H

template <typename T>
class Node{
    public:
    T data;     // VALUE THAT WE NEED TO STORE
    Node* next; // POINTER TO THE NEXT NODE


};

// Global pointer for the top of the stack
template <typename T>
Node<T>* head = nullptr;


//## Operations for the stack

// ISEMPTY
template <typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

// PUSH
template <typename T>
void push(T newData){
    // Create a new node and assign the newData
    Node<T>* newNode = new Node<T>();
    newNode->data = newData;
```

```
34      // New node points to current top, then becomes the new top
35      newNode->next = head<T>;
36      head<T> = newNode;
37  }
38
39  // POP
40  template <typename T>
41  T pop() {
42      if (isEmpty<T>()) {
43          std::cout << "Stack underflow!\n";
44          return T{};
45      }
46
47      // Save the current top node
48      Node<T>* temp = head<T>;
49
50      // Store its data
51      T tempVal = temp->data;
52
53      // Move the head to the next node
54      head<T> = head<T>->next;
55
56      // Free the removed node
57      delete temp;
58
59      // return the removed value
60      return tempVal;
61  }
```

```cpp
// TOP
template <typename T>
void Top(){
    // Error Catching
    if (isEmpty<T>()) {
        std::cout << "Stack underflow!\n";
        return;
    }
    // Display the top node
    std::cout << "Top of Stack:  " << head<T>->data << std::endl;
}


// DISPLAY ALL
template <typename T>
void displayAll(){
    if (isEmpty<T>()) {
        std::cout << "Stack underflow!\n";
        return;
    }

    std::cout << "Stack (top to bottom): ";
    Node<T>* tempVal = head<T>;
    while (tempVal != nullptr) {
        std::cout << tempVal->data;
        if (tempVal->next != nullptr) std::cout << " -> ";
        tempVal = tempVal->next;
    }
    std::cout << std::endl;
}
```

```cpp
// CLEAR (had to add this after answers didn't match and realized I had to clear the stack after every call)
template <typename T>
void clear() {
    while (!isEmpty<T>()) {
        pop<T>();
    }
}

// COMPARATOR
bool comparator(char openSymbol, char closeSymbol){
    // checkes for all three types of brackets, parentheses, and braces
    return (openSymbol == '(' && closeSymbol == ')') || (openSymbol == '{' && closeSymbol == '}') || (openSymbol == '[' && closeSymbol == ']');
}
```

```cpp
// DELIMITERS
bool isOpen(char i){ return i == '(' || i == '{' || i == '['; }
bool isClose(char j){ return j == ')' || j == '}' || j == ']'; }

// PARSER
bool parser(const std::string& expressions){
    clear<char>(); // clear stack at every validation call

    for (char k : expressions){
        if (!isOpen(k) && !isClose(k)) {
            continue; // a) if not balancing continue
        }

        if (isOpen(k)) {
            push<char>(k); // b) if opening symbol, we push em
        }
        else { // c) if closing symbol
            if (isEmpty<char>()) { // c.i) stack empty check
                std::cout << "Err! '" << k << "' has no matching opening symbol.\n";
                return false;
            }
            char sym = pop<char>(); // c.ii) pop the stack if stack not empty
            if (!comparator(sym, k)) { // if popped symbol isn't the same, error em!
                std::cout << "Err! '" << sym << "' does not match '" << k << "'.\n";
                return false;
            }
        }
    }

    if (!isEmpty<char>()) { // 3. if stack is not empty, show error!
        std::cout << "Err! Stack is not empty (perhaps there is an unmatched opening symbol).\n";
        return false;
    }

    return true;
}

#endif
```

(steps are located within the code)

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y	`Expression: (A+B)+(C-D)` `Valid? Y`	All opening symbols have a matching closing symbol of the same type; the stack empties completely and it's considered balanced.

((A+B)+(C-D)	N	`Expression: ((A+B)+(C-D)`
		`Err! Stack is not empty (perhaps there is an unmatched opening symbol).`
		`Valid? N`
			There is an unmatched opener in the expression, leading the parser to catch it and provide the appropriate validity.

((A+B)+[C-D])	Y	Expression: ((A+B)+[C-D]) Valid? Y	Even if the expression mixes the opening and closing symbols, each still has a matching pair with each other.
((A+B]+[C-D]}	N	Expression: ((A+B]+[C-D]} Err! '(' does not match ']'. Valid? N	There is a symbol mismatch between the opener, '(' and the closer, ']' which the parser catches and shows the error.

Tools Analysis:

- How do the different internal representations affect the implementation and usage of the stack?

The different internal representations such as with array or linked list-based types of stacks have a significant impact on implementation as well as its usage. When using an array based stack, we need a predefined, constant maximum size which could potentially lead to a stack overflow if exceeded. On the other hand, a linked list based stack like what we did for our supplementary allows for dynamically manipulating elements without a size limit, though, it uses more memory than with the array-based one.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Proper Code Format**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Had to have the proper code format of cycling through a string list shown to me and a quick refresher on ternary operations as I used booleans in stackList.h for the supplementary.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.